

Major Topics in C#

1. Introduction to C#

- **What is C#?**
 - A high-level, object-oriented programming language developed by Microsoft.
 - Part of the .NET ecosystem, used for building applications like desktop apps, web applications, mobile apps, and games.
 - **Why Learn C#?**
 - Modern features like garbage collection, LINQ, async/await, etc.
 - Strong integration with the Microsoft ecosystem (Windows, Azure, etc.).
-

2. Basic Concepts in C#

Data Types:

- **Value Types:** int, float, double, char, bool, struct
- **Reference Types:** string, array, class, interface, delegate

Code Example:

```
int num = 10; // Value type  
string name = "John"; // Reference type
```

Variables:

- **Declaration and Initialization:** Variables hold data and are declared with a type.
- **Type Inference:** var allows the compiler to infer the type.

Code Example:

```
int x = 5; // Explicit declaration  
var y = 10; // Implicit declaration (compiler infers 'int')
```

Constants:

- **Constants** are values that cannot be changed after being set.

Code Example:

```
const double Pi = 3.14; // Pi will always be 3.14
```

Enums:

- **Enums** define named constant values, making the code more readable.

Code Example:

```
enum Day { Monday, Tuesday, Wednesday }
```

```
Day today = Day.Monday;
```

3. Control Flow

Conditionals:

- **if-else statement:** Used for making decisions based on conditions.

Code Example:

```
if (x > 10)
{
    Console.WriteLine("x is greater than 10");
}
else
{
    Console.WriteLine("x is less than or equal to 10");
}
```

Switch Statement:

- **switch** is used for multiple possible conditions.

Code Example:

```
switch (day)
{
    case "Monday":
        Console.WriteLine("Start of the week");
        break;
    case "Friday":
```

```

        Console.WriteLine("End of the week");
        break;
    default:
        Console.WriteLine("Middle of the week");
        break;
}

```

Loops:

- **for loop:** Loop through a range or collection.

Code Example:

```

for (int i = 0; i < 5; i++)
{
    Console.WriteLine(i); // Output: 0, 1, 2, 3, 4
}

```

- **while loop:** Continues as long as a condition is true.

Code Example:

```

int count = 0;
while (count < 5)
{
    Console.WriteLine(count); // Output: 0, 1, 2, 3, 4
    count++;
}

```

- **foreach loop:** Iterates through a collection.

Code Example:

```

//int[] numbers = { 1, 2, 3, 4, 5 };
or
int[] numbers = [ 1, 2, 3, 4, 5 ];
foreach (int number in numbers)
{

```

```
    Console.WriteLine(number);  
}
```

4. Object-Oriented Programming in C#

Classes and Objects:

- **Class** is a blueprint, and an **Object** is an instance of a class.

Code Example:

```
public class Car  
{  
    public string Model;  
    public int Year;  
  
    public void Drive()  
    {  
        Console.WriteLine($"Driving a {Year} {Model}");  
    }  
}
```

```
Car myCar = new Car();  
myCar.Model = "Tesla";  
myCar.Year = 2023;  
myCar.Drive(); // Output: Driving a 2023 Tesla
```

Methods and Functions:

- **Method**: A block of code that performs a task.

Code Example:

```
public int Add(int a, int b)  
{  
    return a + b;  
}
```

```
}
```

- Calling the method:

```
int result = Add(5, 3); // Output: 8
```

Constructors and Destructors:

- **Constructor** initializes the object when it's created.
- **Destructor** cleans up before an object is destroyed (automated garbage collection in C#).

Code Example:

```
public class Person
{
    public string Name;

    // Constructor
    public Person(string name)
    {
        Name = name;
    }
}

Person p = new Person("John");
```

5. Inheritance, Polymorphism, Encapsulation, and Abstraction

Inheritance:

- A class can inherit methods and properties from another class.

Code Example:

```
public class Animal
{
    public void Eat() { Console.WriteLine("Eating..."); }
}
```

```
public class Dog : Animal
{
    public void Bark() { Console.WriteLine("Barking..."); }
}
```

```
Dog dog = new Dog();
dog.Eat(); // Inherited method
dog.Bark(); // Dog's method
```

Polymorphism:

- **Overriding:** Derived classes can change the behavior of base class methods.

Code Example:

```
public class Animal
{
    public virtual void Speak()
    {
        Console.WriteLine("Animal Sound");
    }
}
```

```
public class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Bark");
    }
}
```

```
Animal myDog = new Dog();  
myDog.Speak(); // Output: Bark
```

Encapsulation:

- Restrict access to class members and provide controlled access through methods.

Code Example:

```
public class BankAccount  
{  
    private double balance;  
  
    public void Deposit(double amount)  
    {  
        if (amount > 0) balance += amount;  
    }  
  
    public double GetBalance()  
    {  
        return balance;  
    }  
}
```

Abstraction:

- Hiding implementation details and exposing only the necessary features.

Code Example:

```
public abstract class Shape  
{  
    public abstract double Area(); // Abstract method  
}
```

```
public class Circle : Shape
{
    public double Radius { get; set; }

    public override double Area() => Math.PI * Radius * Radius;
}
```

6. Collections and Generics

Arrays:

- Fixed-size collection of elements.

Code Example:

```
int[] numbers = { 1, 2, 3, 4, 5 };
Console.WriteLine(numbers[0]); // Output: 1
```

Lists, Dictionaries, and Queues:

- **List:** Dynamic collection, unlike arrays.
- **Dictionary<TKey, TValue>:** Collection of key-value pairs.

Code Example:

```
List<int> numbers = new List<int> { 1, 2, 3 };
numbers.Add(4);
Console.WriteLine(numbers[3]); // Output: 4
```

```
Dictionary<string, int> scores = new Dictionary<string, int>
{
    { "Alice", 85 },
    { "Bob", 90 }
};
Console.WriteLine(scores["Alice"]); // Output: 85
```

Generics:

- Allows you to define type-safe collections or methods.

Code Example:

```
public class Box<T>
{
    public T Item { get; set; }
}

Box<int> integerBox = new Box<int> { Item = 42 };
Console.WriteLine(integerBox.Item); // Output: 42
```

7. Exception Handling**Try-Catch-Finally:**

- Handling runtime errors and ensuring that the final block of code executes.

Code Example:

```
try
{
    int result = 10 / 0;
}
catch (DivideByZeroException ex)
{
    Console.WriteLine("Error: " + ex.Message);
}
finally
{
    Console.WriteLine("This code always runs.");
}
```

8. LINQ (Language Integrated Query)**LINQ to Objects:**

- Perform queries directly on objects in memory using LINQ.

Code Example:

```
var evenNumbers = new List<int> { 1, 2, 3, 4, 5 }.Where(n => n % 2 == 0);  
foreach (var number in evenNumbers)  
{  
    Console.WriteLine(number); // Output: 2, 4  
}
```

LINQ to SQL:

- Query databases using LINQ syntax.

Code Example:

```
var query = from student in context.Students  
            where student.Age > 18  
            select student;
```

9. Asynchronous Programming

Async and Await:

- Allows the program to run non-blocking code asynchronously.

Code Example:

```
public async Task<string> FetchDataAsync()  
{  
    using (var client = new HttpClient())  
    {  
        string data = await client.GetStringAsync("https://api.example.com");  
        return data;  
    }  
}
```

10. Delegates, Events, and Lambdas

Delegates:

- A reference to a method that can be passed around.

Code Example:

```
public delegate void PrintMessage(string message);

PrintMessage msg = message => Console.WriteLine(message);

msg("Hello, World!"); // Output: Hello, World!
```

Events:

- An event allows a class to notify other classes when something happens.

Code Example:

```
public event EventHandler OnEvent;

public void TriggerEvent()
{
    OnEvent?.Invoke(this, EventArgs.Empty);
}
```

Lambdas:

- Inline method definition for brevity.

Code Example:

```
Func<int, int, int> add = (a, b) => a + b;

Console.WriteLine(add(2, 3)); // Output: 5
```

11. File Handling and Serialization

File I/O:

- Reading from and writing to files.

Code Example:

```
using (StreamWriter writer = new StreamWriter("file.txt"))
{
    writer.WriteLine("Hello, World!");
}
```

```
}
```

```
using (StreamReader reader = new StreamReader("file.txt"))  
  
{  
    Console.WriteLine(reader.ReadToEnd()); // Output: Hello, World!  
}
```

Serialization and Deserialization:

- Converting objects to and from JSON/XML.

Code Example (JSON):

```
public class Person  
{  
    public string Name { get; set; }  
    public int Age { get; set; }  
}  
  
var person = new Person { Name = " Alice", Age = 30 };  
string json = JsonConvert.SerializeObject(person);  
Console.WriteLine(json);
```

Output:

```
{"Name":"Alice","Age":30}
```

12. Memory Management

Garbage Collection:

- Automatic memory management in C#.

Explanation: C# uses a garbage collector (GC) to manage memory automatically, releasing memory that is no longer in use.

IDisposable and Using Statement:

- Cleaning up resources like file streams, database connections.

Code Example:

```
using (var resource = new Resource())  
{  
    resource.Use();  
}
```

13. Multithreading

Threads:

- Running code concurrently in different threads.

Code Example:

```
Thread t = new Thread(() => Console.WriteLine("Hello from another thread"));  
t.Start();
```

Synchronization:

- Ensuring thread safety with lock.

Code Example:

```
private static readonly object lockObj = new object();  
  
public void Method()  
{  
    lock (lockObj)  
    {  
        // Thread-safe code  
    }  
}
```

14. C# Features

Nullable Types:

- Allow value types to be null.

Code Example:

```
int? x = null;

if (x.HasValue)
{
    Console.WriteLine(x.Value);
}
```

Tuples:

- Storing multiple values of different types.

Code Example:

```
var person = (Name: "John", Age: 30);

Console.WriteLine(person.Name); // Output: John
```

Pattern Matching:

- Matching complex data types in switch or if.

Code Example:

```
object obj = "Hello";

switch (obj)
{
    case string s:
        Console.WriteLine("It's a string");
        break;
    default:
        Console.WriteLine("Unknown type");
        break;
}
```

15. Design Patterns and Unit Testing

Design Patterns:

- Reusable solutions to common problems in software design.

Code Example (Singleton):

```
public sealed class Singleton2
{
    // Step 1: Private static readonly instance
    private static readonly Singleton2 _instance = new Singleton2();

    // Step 2: Private constructor to prevent external instantiation
    private Singleton2()
    {
        Console.WriteLine("Singleton instance created.");
    }

    // Step 3: Public static property to access the instance
    public static Singleton2 Instance
    {
        get
        {
            return _instance;
        }
    }

    // Sample method
    public void Log(string message)
    {
        Console.WriteLine("Log: " + message);
    }
}

class Program
{
    public static void Main()
    {
        var obj1 = Singleton2.Instance;
        obj1.Log("First call");

        var obj2 = Singleton2.Instance;
        obj2.Log("Second call");

        Console.WriteLine(object.ReferenceEquals(obj1, obj2));
    }
}
```

```
}  
}
```

Unit Testing:

- Writing tests to verify the correctness of code.

Code Example (xUnit):

// File: CalculatorTests.cs

using Xunit;

public class CalculatorTests

{

private readonly Calculator _calc;

public CalculatorTests()

{

_calc = new Calculator(); // Arrange

}

[Fact]

public void Add_TwoNumbers_ReturnsSum()

{

// Act

var result = _calc.Add(2, 3);

// Assert

Assert.Equal(5, result);

}}

16. Conclusion

- **Recap of Topics Covered:** We covered everything from basic syntax to advanced OOP and asynchronous programming in C#.
- **Importance of C#:** Essential for .NET applications, including desktop apps, web apps, and more.
- **Resources for Further Learning:**
 - Books: *C# in Depth* by Jon Skeet
 - Official Documentation: [Microsoft C# Documentation](#)
- **Q&A**
